

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении практической работы №6
по дисциплине «Функциональное программирование»
Вариант №2.

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Моторин Д.Е.

«_____» _____ 2025г.

Санкт-Петербург, 2025

Содержание

Введение	3
1 Постановка задачи	4
2 Функции и тестирование	5
2.1 Типы данных	5
2.2 Реализация функций	5
2.3 Тестирование	6
2.3.1 Тестирование isCongruent	6
2.3.2 Тестирование treeDepth	6
3 Проверка корректности	8
Выводы	9
Список использованных источников	10
Приложение 1 «Исходный код программы»	11

Введение

Числа a и b считаются равными по модулю числа c , если числа a и b имеют одинаковые остатки при делении на c . При этом c не может быть равно нулю, так как операция деления на ноль не определена.

Бинарное дерево - это структура данных, в которой каждый узел имеет значение и ссылки на правое и левое поддеревья, каждое из которых или является пустым множеством, или бинарным деревом. Узел, не имеющий предшественников, называется корнем дерева. Узлы, не имеющие потомков, называются листьями. Глубина дерева - это максимально длинный путь от корня к листу.

1 Постановка задачи

Цель: реализация автоматического тестирования двух функций.

Задачи:

1. Написать функцию проверки равенства чисел по модулю.
2. Написать функцию вычисления глубины дерева `treeDepth`.
3. Проверить свойства функций с помощью `QuickCheck`.

2 Функции и тестирование

2.1 Типы данных

Тип `Tree` представляет бинарное дерево, где `a` - тип значения в узле (может быть или листом, или новым поддеревом). Тип получает возможность сравнения и вывода на печать.

Листинг 1. Тип `Tree` и конструкторы

```
1 data Tree a = Empty | Node a (Tree a) (Tree a)
2 deriving (Show, Eq)
3
4 emptyTree :: Tree a
5 emptyTree = Empty
6
7 singleNodeTree :: a -> Tree a
8 singleNodeTree x = Node x Empty Empty
9
10 twoBranchTree :: a -> Tree a -> Tree a -> Tree a
11 twoBranchTree x left right = Node x left right
```

Тип `Tree` имеет несколько конструкторов: `emptyTree` строит пустое дерево, `singleNodeTree` строит дерево из одного узла, `twoBranchTree` - узел с заданными левым и правым поддеревьями.

2.2 Реализация функций

Функция `isCongruent` проверяет, равны ли два числа по модулю третьего числа. При этом если модуль равен нулю, функция возвращает `false`, так как деление на ноль не определено.

Листинг 2. Функция `isCongruent`

```
1 isCongruent :: Int -> Int -> Int -> Bool
2 isCongruent a b m
3 | m == 0 = False
4 | otherwise = a `mod` m == b `mod` m
```

Функция `treeDepth` предназначена для вычисления глубины бинарного дерева.

Листинг 3. Функция `treeDepth`

```
1 treeDepth :: Tree a -> Int
2 treeDepth Empty = 0
3 treeDepth (Node _ left right) = 1 + max (treeDepth left) (treeDepth right)
```

Она рекурсивно проходит по всему дереву, вычисляя самый длинный путь от корня до узла. При этом значение узла игнорируется.

2.3 Тестирование

При тестировании используется библиотека QuickCheck, который позволяет быстро генерировать наборы тестов на основе проверки выполнения определенных свойств. Все типы, для которых QuickCheck может автоматически создавать значения, должны быть представителями класса Arbitrary.

2.3.1 Тестирование isCongruent

Свойство prop_modDif утверждает, что если два числа равны по модулю, то их разность делится на модуль. При этом исключаются случаи, когда модуль равен нулю. Свойство prop_symm проверяет, что равенство по модулю является симметричным.

Листинг 4. Свойства функции isCongruent

```
1 prop_modDif :: Int -> Int -> Int -> Property
2 prop_modDif a b m =
3 m /= 0 ==> isCongruent a b m == (mod (a - b) m == 0)
4
5 prop_symm :: Int -> Int -> Int -> Property
6 prop_symm a b m =
7 m /= 0 ==> isCongruent a b m == isCongruent b a m
8
9 prop_eqNum :: Int -> Int -> Property
10 prop_eqNum a m =
11 m /= 0 ==> isCongruent a a m == True
```

Свойство prop_eqNum утверждает, что если одно число равно другому, то они равны по любому ненулевому модулю.

2.3.2 Тестирование treeDepth

По умолчанию для типа Tree нет экземпляра Arbitrary, поэтому необходимо прописать его явно. Дерево рекурсивно генерируется, для каждого узла случайно определяется, является ли оно листом. Также реализован shrink, который упрощает минимальный пример, вызывающий ошибку.

Листинг 5. Генератор произвольных деревьев

```
1 instance Arbitrary a => Arbitrary (Tree a) where
2 arbitrary = sized genTree
3 where
4   genTree n
5   | n <= 0 = return Empty
6   | otherwise = do
7     isEmpty <- arbitrary :: Gen Bool
8     if isEmpty
9     then return Empty
```

```

10 else do
11   x <- arbitrary
12   left <- genTree (n-1)
13   right <- genTree (n-1)
14   return $ Node x left right
15   shrink Empty = []
16   shrink (Node x left right) =
17     [Empty] ++
18     [Node x left' right | left' <- shrink left] ++
19     [Node x left right' | right' <- shrink right] ++
20     [Node x' left right | x' <- shrink x] ++
21     [left, right]

```

Свойство `prop_emptyDepth` проверяет, что глубина пустого дерева равна нулю. Свойство `prop_sNodeDepth` проверяет, что глубина дерева с одним узлом равна единице.

Листинг 6. Свойства функции `treeDepth`

```

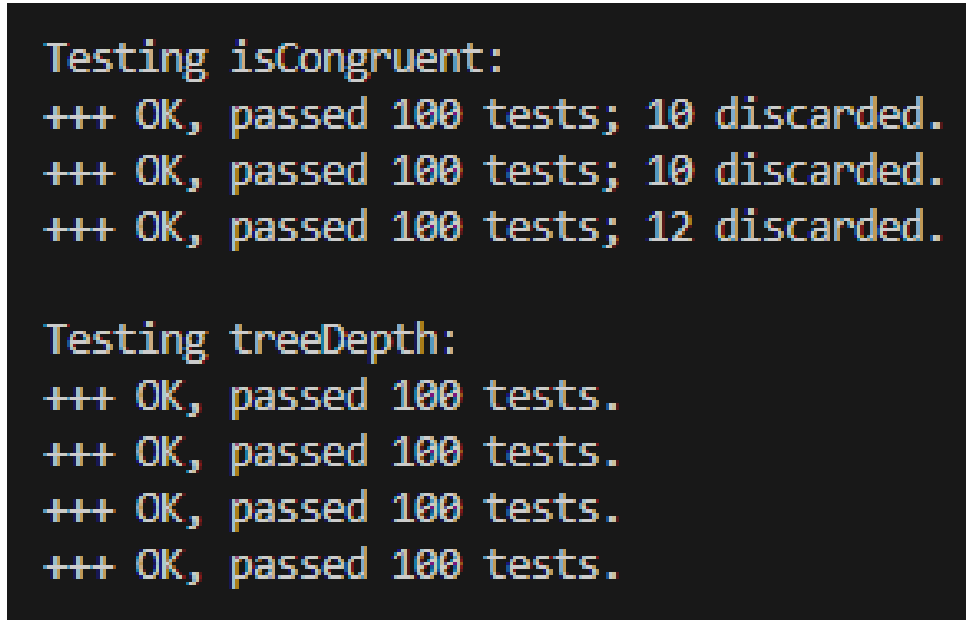
1 prop_emptyDepth :: Bool
2 prop_emptyDepth = treeDepth (Empty :: Tree Int) == 0
3
4 prop_sNodeDepth :: Int -> Bool
5 prop_sNodeDepth x =
6   treeDepth (Node x Empty Empty) == 1
7
8 prop_twoBranchDepth :: Int -> Tree Int -> Tree Int -> Bool
9 prop_twoBranchDepth x left right =
10   let depthLeft = treeDepth left
11       depthRight = treeDepth right
12       expectedDepth = 1 + max depthLeft depthRight
13   in treeDepth (Node x left right) == expectedDepth
14
15 prop_depthConst :: Int -> Tree Int -> Bool
16 prop_depthConst x tree =
17   let originalDepth = treeDepth tree
18       newTree = Node x tree Empty
19   in treeDepth newTree >= originalDepth

```

Свойство `prop_twoBranchDepth` проверяет, что глубина дерева с двумя ветвями равна максимальной глубине среди ветвей плюс один. Свойство `prop_depthConst` проверяет, что добавление узла в дерево не уменьшает его глубину.

3 Проверка корректности

На рисунке 1 приведен результат работы тестов для функций `isCongruent` и `treeDepth`.



```
Testing isCongruent:
+++ OK, passed 100 tests; 10 discarded.
+++ OK, passed 100 tests; 10 discarded.
+++ OK, passed 100 tests; 12 discarded.

Testing treeDepth:
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.
+++ OK, passed 100 tests.
```

Рис. 1. Результаты тестирования

Тестирование подтвердило, что программа работает корректно. Часть тестов функции `isCongruent` отброшены, так как не удовлетворяют условию ненулевого модуля.

Выводы

В ходе работы реализована программа, которая автоматически тестирует заданные функции. Написана функция проверки равенства чисел по модулю `isCongruent`. Написана функция определения глубины бинарного дерева `treeDepth`. Для обеих функций проверено выполнение всех заданных свойств с помощью библиотеки `QuickCheck`. Тестирование программы показало, что все функции работают корректно и обрабатывают частные случаи.

Список литературы

- [1] Курт У. «Программируй на Haskell». М.: Издательство, 2019. 320 стр.
- [2] Липовача М. «Изучай Haskell во имя добра». М.: ДМК Пресс, 2012. 490 стр.

Приложение 1 «Исходный код программы»

Листинг 7. Код работы

```
1 import Test.QuickCheck
2 import Lib
3
4 -- isCongruent
5 prop_modDif :: Int -> Int -> Int -> Property
6 prop_modDif a b m =
7 m /= 0 ==> isCongruent a b m == (mod (a - b) m == 0)
8
9 prop_symm :: Int -> Int -> Int -> Property
10 prop_symm a b m =
11 m /= 0 ==> isCongruent a b m == isCongruent b a m
12
13 prop_eqNum :: Int -> Int -> Property
14 prop_eqNum a m =
15 m /= 0 ==> isCongruent a a m == True
16
17 -- генератор произвольных деревьев
18 instance Arbitrary a => Arbitrary (Tree a) where
19 arbitrary = sized genTree
20 where
21 genTree n
22 | n <= 0 = return Empty
23 | otherwise = do
24 isEmpty <- arbitrary :: Gen Bool
25 if isEmpty
26 then return Empty
27 else do
28 x <- arbitrary
29 left <- genTree (n-1)
30 right <- genTree (n-1)
31 return $ Node x left right
32 shrink Empty = []
33 shrink (Node x left right) =
34 [Empty] ++
35 [Node x left' right | left' <- shrink left] ++
36 [Node x left right' | right' <- shrink right] ++
37 [Node x' left right | x' <- shrink x] ++
38 [left, right]
39
40 -- treeDepth
41 prop_emptyDepth :: Bool
42 prop_emptyDepth = treeDepth (Empty :: Tree Int) == 0
43
44 prop_sNodeDepth :: Int -> Bool
45 prop_sNodeDepth x =
46 treeDepth (Node x Empty Empty) == 1
47
48 prop_twoBranchDepth :: Int -> Tree Int -> Tree Int -> Bool
49 prop_twoBranchDepth x left right =
50 let depthLeft = treeDepth left
51 depthRight = treeDepth right
52 expectedDepth = 1 + max depthLeft depthRight
```

```

53 in treeDepth (Node x left right) == expectedDepth
54
55 prop_depthConst :: Int -> Tree Int -> Bool
56 prop_depthConst x tree =
57   let originalDepth = treeDepth tree
58   newTree = Node x tree Empty
59   in treeDepth newTree >= originalDepth
60
61
62 main :: IO ()
63 main = do
64   putStrLn "Testing isCongruent:"
65   quickCheck prop_modDif
66   quickCheck prop_symm
67   quickCheck prop_eqNum
68
69   putStrLn "\nTesting treeDepth:"
70   quickCheck prop_emptyDepth
71   quickCheck prop_sNodeDepth
72   quickCheck prop_twoBranchDepth
73   quickCheck prop_depthConst
74
75 module Lib (
76   isCongruent,
77   Tree(..),
78   treeDepth
79 ) where
80
81 isCongruent :: Int -> Int -> Int -> Bool
82 isCongruent a b m
83 | m == 0 = False
84 | otherwise = a `mod` m == b `mod` m
85
86 data Tree a = Empty | Node a (Tree a) (Tree a)
87 deriving (Show, Eq)
88
89 treeDepth :: Tree a -> Int
90 treeDepth Empty = 0
91 treeDepth (Node _ left right) = 1 + max (treeDepth left) (treeDepth
    right)
92
93 emptyTree :: Tree a
94 emptyTree = Empty
95
96 singleNodeTree :: a -> Tree a
97 singleNodeTree x = Node x Empty Empty
98
99 twoBranchTree :: a -> Tree a -> Tree a -> Tree a
100 twoBranchTree x left right = Node x left right

```